IES NERVIÓN · SEVILLA

EXAMEN DE PROGRAMACIÓN · UNIDAD DIDÁCTICA 2

Programación · 1.º DAW · RA3: estructuras de control

Fecha: 13/11/2026 | Duración: 2 horas y media | Lenguaje: Java

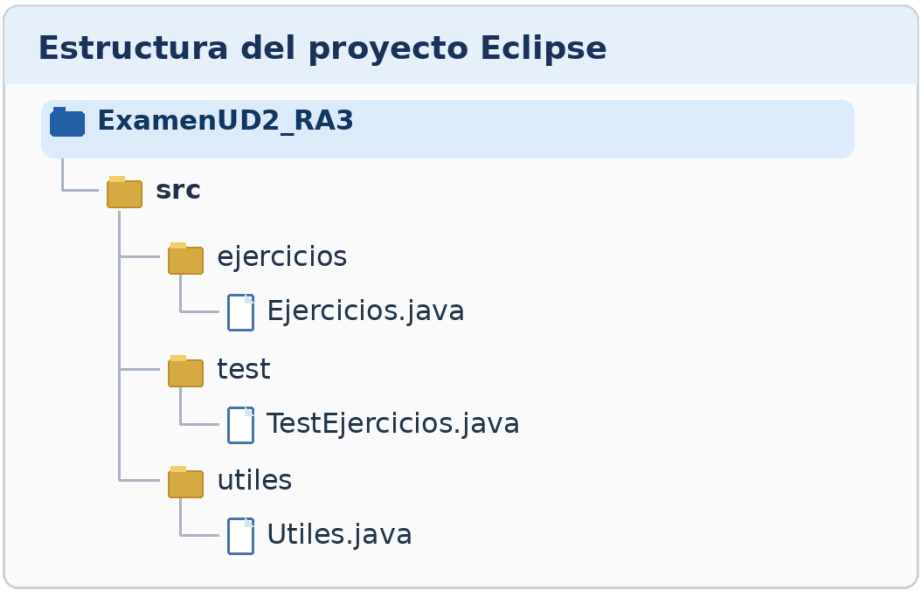
Alumno/a: Luján Martinez Fernandez Grupo: _____

INSTRUCCIONES GENERALES

Los seis ejercicios obligatorios tienen exactamente el mismo valor. La evaluación se vincula a los criterios del RA3 indicados en cada ejercicio. El ejercicio 7, en inglés, es voluntario y solo puede subir la calificación, nunca bajarla.

Crea en Eclipse un proyecto Java llamado ExamenUD2_RA3, con los paquetes ejercicios, test y utiles, siguiendo la estructura del esquema. Implementa cada ejercicio mediante un método independiente en la clase Ejercicios y respeta exactamente las cabeceras propuestas. Puedes crear métodos auxiliares.

Crea la clase TestEjercicios con el método main dedicado exclusivamente a las pruebas.



Puedes consultar tus propios proyectos de clase, pero no Internet. No se exige lectura por teclado ni se evaluarán las excepciones. No utilices continue. Procura realizar un único return final en cada método que devuelva un valor.

EJERCICIO 1. Clasificación de triángulos

Criterios de evaluación RA3: a) estructuras de selección; e) programas ejecutables con estructuras de control. (Séneca: a, f)

Recibes tres longitudes enteras positivas. Debes decidir si pueden formar un triángulo y, si es posible, clasificarlo. Un triángulo solo es válido cuando la suma de dos lados cualesquiera es mayor que el tercero. Si no se cumple esa condición, el resultado será "INVALIDO".

Para un triángulo válido, devuelve "RECTANGULO" si cumple el teorema de Pitágoras. Si no es rectángulo, devuelve "EQUILATERO" cuando los tres lados son iguales, "ISOSCELES" cuando exactamente dos son iguales y "ESCALENO" cuando todos son distintos.

```
public static String clasificarTriangulo(Integer a, Integer b, Integer c)
```

Ejemplos: clasificarTriangulo(3, 4, 5) → "RECTANGULO"; clasificarTriangulo(5, 5, 5) → "EQUILATERO"; clasificarTriangulo(5, 5, 8) → "ISOSCELES"; clasificarTriangulo(4, 6, 9) → "ESCALENO"; clasificarTriangulo(2, 2, 4) → "INVALIDO".

EJERCICIO 2. Triángulo de números alternos

Criterios de evaluación RA3: b) estructuras de repetición; g) comentario y documentación del código. (Séneca: b, i)

Escribe un método que reciba como parámetro un entero n y dibuje un triángulo de números 1 y 0. La primera fila contiene un solo número; cada fila siguiente tiene un número más que la anterior. Los números de cada fila se alternan (1, 0, 1, 0...), y el primer número de una fila debe ser distinto del primero de la fila anterior.

Para n = 5, la salida debe ser exactamente:

```
1
01
101
0101
10101
```

Para n = 1 se imprime únicamente 1. Para n = 0 no se imprime nada. Puedes suponer que n nunca es negativo.

Añade comentarios breves que expliquen la finalidad de cada bucle y de la condición utilizada.

```
public static void trianguloAlterno(Integer n)
```

EJERCICIO 3. Buscar un número por sus divisores

Criterios de evaluación RA3: b) estructuras de repetición; c) sentencias de salto. (Séneca: b, c)

Los divisores propios de un número son todos menos él mismo. Por ejemplo, los divisores propios de 12 son 1, 2, 3, 4 y 6 (cinco divisores). Recorre los enteros del intervalo [inicio, fin] en orden ascendente y busca el PRIMER número que tenga al menos minimoDivisores divisores propios.

Puedes suponer que inicio <= fin y que minimoDivisores es positivo. Si ninguno cumple la condición, devuelve -1.

```
public static Integer primeroConDivisores(Integer inicio, Integer fin, Integer minimoDivisores)
```

Ejemplos: primeroConDivisores(10, 20, 4) → 12; primeroConDivisores(2, 10, 3) → 6; primeroConDivisores(13, 13, 2) → -1; primeroConDivisores(10, 10, 3) → 10.

EJERCICIO 4. Tribonacci recursivo

Criterios de evaluación RA3: e) programas ejecutables con estructuras de control; g) comentario y documentación del código. (Séneca: f, i)

La sucesión de Tribonacci es parecida a Fibonacci, pero cada término se obtiene sumando los TRES anteriores. Sus tres primeros valores son T(0) = 0, T(1) = 1 y T(2) = 1. A partir de ahí, T(n) = T(n-1) + T(n-2) + T(n-3), para n >= 3.

Por tanto, los primeros términos son: 0, 1, 1, 2, 4, 7, 13, 24, 44... Por ejemplo, T(5) = T(4) + T(3) + T(2) = 4 + 2 + 1 = 7.

Escribe un método que reciba la posición n (comenzando en 0) y devuelva el término correspondiente de la sucesión. Debe resolverse obligatoriamente mediante recursividad, como el Fibonacci visto en clase. Puedes suponer que n está entre 0 y 12.

Comenta brevemente el código.

```
public static Integer tribonacci(Integer n)
```

Ejemplos: tribonacci(0) → 0; tribonacci(1) → 1; tribonacci(2) → 1; tribonacci(3) → 2; tribonacci(5) → 7; tribonacci(7) → 24.

EJERCICIO 5. Lista casi creciente

Criterios de evaluación RA3: a) estructuras de selección; f) prueba y depuración de programas. (Séneca: a, g)

Una lista de enteros es casi creciente cuando, al recorrerla de izquierda a derecha, sus valores aumentan, pero se permite como máximo una ruptura. Se produce una ruptura si un elemento es menor o igual que el anterior. Por tanto, dos valores consecutivos iguales también cuentan como ruptura.

Escribe un método que devuelva true si la lista tiene cero o una ruptura, y false si tiene dos o más. Puedes suponer que la lista no contiene elementos null. Una lista vacía o con un solo elemento se considera casi creciente.

Incluye al menos dos pruebas propias distintas de los ejemplos: indica en comentarios el resultado obtenido.

```
public static Boolean esCasiCreciente(List<Integer> datos)
```

Ejemplos: esCasiCreciente(List.of(2, 4, 6, 8)) → true; esCasiCreciente(List.of(2, 5, 3, 7, 9)) → true; esCasiCreciente(List.of(2, 5, 3, 7, 4)) → false; esCasiCreciente(List.of(1, 3, 3, 5)) → true.

EJERCICIO 6. Primera ventana que alcanza el objetivo

Criterios de evaluación RA3: c) sentencias de salto; f) prueba y depuración de programas. (Séneca: c, g)

Una lista contiene las ventas diarias de una tienda. Queremos encontrar el primer grupo de longitud días consecutivos cuya suma sea mayor o igual que objetivo. Cada grupo se llama ventana. Devuelve la posición (empezando en 0) del primer día de la primera ventana válida.

Ejemplo: ventas = [2, 1, 5, 3, 2], longitud = 3 y objetivo = 9.

Primera ventana, posiciones 0 a 2: [2, 1, 5] → suma 8 (no vale).

Segunda ventana, posiciones 1 a 3: [1, 5, 3] → suma 9 (sí vale).

Resultado: 1, porque la primera ventana válida empieza en la posición 1.

Devuelve -1 si ninguna ventana alcanza el objetivo. Los valores de la lista son no negativos y no contienen null y longitud se supone que es >=1 pero menor o igual que el tamaño de la lista.

Comprueba el método desde TestEjercicios mediante al menos dos pruebas propias, distintas de los ejemplos: una con coincidencia y otra sin coincidencia. Anota en comentarios la entrada y el resultado esperado y muestra el resultado obtenido.

```
public static Integer primeraVentanaObjetivo(List<Integer> ventas, Integer longitud, Integer objetivo)
```

Ejemplos: primeraVentanaObjetivo(List.of(2, 1, 5, 3, 2), 3, 9) → 1; primeraVentanaObjetivo(List.of(2, 1, 5, 3, 2), 3, 11) → -1; primeraVentanaObjetivo(List.of(4, 4, 1, 6), 2, 8) → 0.

OPTIONAL ENGLISH BONUS

EXERCISE 7. Even and odd numbers

RA3 assessment criteria: b) repetition structures; f) program testing and debugging. (Séneca: b, g)

Write a method that checks whether all consecutive numbers in a list alternate between even and odd. Return true if they always alternate, and false if two consecutive numbers are both even or both odd.

For example, [2, 3, 8, 5] returns true because the numbers alternate even, odd, even, odd. However, [2, 3, 5, 8] returns false because 3 and 5 are both odd.

Return true for an empty list or a list with only one element. Use English names for all variables. Assume that the numbers are non-negative and not null.

```
public static Boolean alternatesEvenOdd(List<Integer> numbers)
```

Examples: alternatesEvenOdd(List.of(2, 3, 8, 5)) → true; alternatesEvenOdd(List.of(1, 4, 7, 10)) → true; alternatesEvenOdd(List.of(2, 3, 5, 8)) → false; alternatesEvenOdd(List.of(2, 4, 7, 8)) → false; alternatesEvenOdd(List.of(3)) → true; alternatesEvenOdd(List.of()) → true.

This optional exercise can only improve your grade; not submitting it or making mistakes in it will never reduce your grade.

ENTREGA

Al terminar, exporta el proyecto en un solo ZIP y entrégalo con el nombre exacto apellidosnombreexamenUD2.zip, sustituyendo apellidosnombre por tus apellidos y nombre, sin espacios ni tildes.